溯真 ReproLab · 技术文档

溯真 ReproLab · 技术文档

> 对应省赛要求 2.1–2.3：**意图控制策略** / **架构设计** / **可复现性论证**。全文 ≤30 页。 作品：溯真 ReproLab　组别：B 组　学校：福州理工学院　作者：王钰森 仓库：https://github.com/9568138-cloud/reprolab（43 个提交 / 27 个 `[iter N]` / 39 条测试全绿）

一、作品与技术目标

ReproLab 要解决的不是”让人更容易做出统计”，而是**让统计结论可被他人复核**。因此技术目标被写成三条可检验的约束，全文所有设计都服务于它们：

#	技术目标	可检验的判据
G1	AI 的输出必须**有界**	模型只能产出白名单算子的 JSON 计划，越界抛错；计划可导出、可审阅
G2	执行必须**确定**	同一 `（数据，计划）` 两次执行，产物总哈希逐位一致
G3	结论必须**可第三方验证**	复现包解压后不依赖 ReproLab，一条 `node verify.mjs` 判定通过/失败

反例即设计约束：任何削弱 G1–G3 的实现都不可接受。例如”让模型直接写 pandas 代码最省事”，但它同时破坏 G1（无界）、G2（措辞不同结果不同）、G3（评委无法在不信任环境里复算），所以被否掉。

二、意图控制策略：如何让 AI 的输出可被机器校验

2.1 问题

让大模型自由写代码并执行，会同时牺牲三件事：

- **可复现性**：模型每次生成的代码不同，结果不同；
- **安全性**：需要在服务端执行用户/模型的代码，沙箱逃逸面巨大；
- **可评审性**：评委看不懂生成逻辑，也就无法判断结论是否可信。

2.2 做法一：把 AI 的输出压缩到「计划 DSL」

AI 只被允许输出一个固定结构、可枚举的 JSON：

```
{ "version": 1, "question": "新建住宅与二手住宅指数是否相关",
  "steps": [
    { "id": "s1", "op": "describe", "params": { "column": "新建住宅环比" } },
    { "id": "s2", "op": "filter_rows", "params": { "column": "城市", "cmp": "!=", "value": "北京" } },
    { "id": "s3", "op": "correlate", "params": { "x": "新建住宅环比", "y": "二手住宅环比" } }
  ] }
```

算子白名单 13 个:

算子	作用	是否改动数据
`describe`	描述统计（n / 均值 / 标准差 / 分位）	否
`filter_rows`	条件筛选（先筛后统计，支持数值与文本列）	**是**
`drop_outliers`	按 IQR 剔除异常点	**是**
`transform_log`	对数变换	**是**
`normalize`	z-score 标准化	**是**
`correlate`	皮尔逊相关（成对完整样本）	否
`fit_linear`	最小二乘拟合（斜率/截距/R^2）	否
`scatter`	散点图（含被剔除点标红）	否
`group_summary`	分组汇总（按基数自动选分组列）	否
`ttest`	Welch t 检验（两组比较）	否
`trend`	时间趋势（趋势线 + R^2）	否
`histogram`	分布直方图	否
`missing_report`	缺失盘点	否

校验器 `validatePlan` 负责：算子是否在白名单、参数类型是否正确、列名是否存在、步数是否 ≤12、破坏性算子是否带 `confirm: true`。任何一条不满足 → 抛错，**不执行。**

2.3 做法二：人工确认闸门

改动数据的算子（`filter_rows` / `drop_outliers` / `transform_log` / `normalize`）必须带 `confirm: true`。这道闸门做三次：

1. 计划层：`validatePlan` 缺 `confirm` 直接拒绝；
2. 界面层：不勾选”我确认”复选框，执行按钮保持 `disabled`；
3. 归档层：复现包里的 `verify.mjs` 会再查一遍 `confirm` 标记。

2.4 做法三：结论与统计量强绑定

结论文本不是模型自由生成的散文，而是**受限模板 + 占位符**:

```
"{{x}} 与 {{y}} 呈中等正相关（r = {{r}}, n = {{n}}）"
```

`renderClaim` 在执行结果的对应步骤输出里查找 `r` / `n` 字段；**找不到直接抛错**，页面拒绝渲染该结论。这条从机制上杜绝”AI 编数字”——数字必须来自执行结果，不能来自语言模型。

2.5 做法四：三层降级，保证永远能演示

场景	行为	可复现性
配了 API Key，模型正常	LLM 编译 → 校验 → 执行	计划一旦生成，执行逐位一致
LLM 返回非法 DSL	校验失败 → 自动降级为本地规则编译	同样可复现
完全离线、无 Key	规则编译（关键词 → 算子映射）	同样可复现
用户不愿用 AI	**点选模式**（11 个意图按钮 + 3 个列下拉）	生成同一套 DSL，计划卡标注”没有 AI 参与，完全确定”

2.6 上下文控制

- 送给模型的是**结构化列信息**（列名、类型、行数、缺失率），不是原始数据 → 上下文短、隐私风险低；
- `temperature: 0`；要求只输出 JSON；服务端支持 `response_format: json_object` 时启用；
- 模型输出**永远不直接进入执行层**，必须过校验器。

三、架构设计

3.1 分层

```
┌──────────────────────────────────────────────┐
│  UI 层     index.html + src/ui/app.js          │
│  五段式工作台 · 内联提示 · 图表下钻 · 结论徽章  │
└──────────────────────────────────────────────┘
                │ 纯函数 + 结构化对象
┌──────────────────────────────────────────────┐
│  计划层    src/planner.js    src/audit.js       │
│  自然语言→DSL · 结论卡生成与校验 · 审计判定    │
└──────────────────────────────────────────────┘
                │ plan（JSON）
┌──────────────────────────────────────────────┐
│  内核层    src/kernel/{csv,stats,ops,index,profile} │
│  解析 · 13 个算子 · 逐步执行 · 哈希留痕 · 数据体检 │
└──────────────────────────────────────────────┘
                │ (plan, run) 快照
┌──────────────────────────────────────────────┐
│  导出层    src/export/{zip,package,verify-runner} │
│  手写 ZIP · 复现包 · 自带 verify.mjs · 界面自校验 │
└──────────────────────────────────────────────┘
```

3.2 模块职责与规模

模块	行数	职责	依赖
`kernel/csv.js`	57	CSV 解析、列类型推断、缺失归一	无
`kernel/hash.js`	24	稳定序列化 + SHA-256（浏览器/Node 同结果）	WebCrypto / node:crypto
`kernel/stats.js`	177	描述统计、皮尔逊 r、最小二乘、Welch t、p 值近似、IQR	无
`kernel/ops.js`	293	13 个算子的纯函数实现	stats
`kernel/profile.js`	92	数据体检（缺失率 / 唯一值 / 范围 / 常量列）	无
`kernel/index.js`	122	计划校验、逐步执行、哈希留痕、`canonicalArtifact`	以上
`planner.js`	310	规则编译、LLM 编译、结论卡生成与校验	kernel
`audit.js`	151	审计：逐条重算、三态判定、报告导出	kernel
`export/*.js`	299	ZIP 写入器、复现包组装、界面自校验	无
`ui/app.js`	1,312	工作台交互与渲染	全部
合计	**2,837**	零第三方运行依赖	

关键约束：

- 内核层**不依赖 DOM** → 可在 Node 里直接跑 39 条单测；
- 导出层**不读内核内部状态**，只接受 `(plan, run)` 快照 → 可测试、可替换；
- 层间只有纯函数与可序列化对象 → 没有隐式全局状态，没有时序耦合。

3.3 关键取舍

取舍	选择	理由
框架	不上 React/Vite，原生 ES module	零依赖、无构建产物、无环境漂移；“10 分钟复现”优先于”框架现代”
执行位置	浏览器内	无服务端执行用户代码 → 无边执行漏洞；数据不出本机
数值实现	自研约 200 行统计内核	去掉 numpy/scipy，浏览器与 Node 结果一致，才谈得上逐位复现
ZIP	手写 store 模式写入器	不引第三方包，导出包任何解压工具都能开

取舍	选择	理由
复现口径	只保留一处实现 `canonicalArtifact`	界面重跑、导出包、第三方校验都调它，避免"自家脚本判自家包 FAIL"（这是真实踩过的坑）

3.4 哈希链

```
data.csv ──SHA-256──▸ dataHash          （原始层，只读）
                        │
step1: inputHash=H(rows, cols, excludedRows, transforms) ─▸ outputHash
step2: inputHash=H(step1 输出 + 变更)                    ─▸ outputHash
   ⋮
artifactHash = SHA-256( 稳定序列化( plan + outputs + excludedRows ) )
```

稳定序列化（键排序、数值规范化）保证键序无关 → 同一语义必得同一哈希。`outputHash` 用的是被裁剪到固定精度后的数值序列，避免浮点打印差异造成假失败。

四、可复现性论证

4.1 论证结构（五条链）

1. **输入可固定**：数据只读载入并立即记录 SHA-256；计划是显式 JSON，可导出、可粘贴、可导入（导入时重新过白名单）。
2. **过程可固定**：无随机数、无时间依赖、无并发；算子实现是纯函数；执行顺序由计划的 `steps` 数组唯一决定。
3. **输出可验证**：界面内「重跑一致性校验」对同一 `(data, plan)` 重算并逐位比较产物总哈希，直接给出 ✅/❌ 与两个哈希值。
4. **可第三方验证**：复现包含 `data.csv`、`plan.json`、`run.json`（每步哈希 + 排除行清单）、`report.md`、`meta.json`、`chart.svg`。
5. **可离线自证**：包内自带零依赖 `verify.mjs`，不需要 ReproLab、不需要联网。

```
unzip reprolab-*.zip -d check && cd check && node verify.mjs
# PASS  数据 SHA-256 与包内记录一致          [352e614479276cc6…]
# PASS  产物哈希可重算（执行记录未被篡改）    [af3418573d71ba96…]
# PASS  meta.json 与 run.json 的产物哈希一致
# PASS  计划只使用白名单算子
# PASS  破坏性算子已人工确认
# ✓ 全部通过：数据未改动、执行记录可重算、计划在白名单内。
```

反向验证：把 `run.json` 里任何一个数字（例如 `line.slope`）乘 1.01，`verify.mjs` 立刻 FAIL 并以退出码 1 结束——已实测。界面里也有同口径的「在界面里自校验复现包」，5 项逐项 PASS/FAIL。

4.2 实测证据（2026-10-27 / 28）

项目	结果
重跑一致性	同一数据 + 同一计划执行两次，产物哈希逐位一致
篡改检测	改 `run.json` 任一数字 → FAIL、退出码 1
中文数据（主演示）	国统局 70 城房价指数环比，16,800 行；新建 vs 二手住宅 $r = 0.6234$（$n = 16{,}800$, $p < 0.0001$）；按城市分组 70 组；载入 + 三步执行 34 ms
条件筛选	16,800 行筛掉 15,321，保留 1,479 行，统计口径随之变化且可复算
审计（5 条结论）	3 通过 / 1 不符（报告写 71 个城市，重算 70）/ 1 无法复现（引用不存在的步骤）
审计查正文	正文写 0.5、字段值 0.962865 → 判不符
英文数据	iris 150 行（setosa vs versicolor 花瓣长均值差 −2.798 cm, $p<0.0001$）；airquality 153 行 Ozone 缺失 37 个（24.2%），相关样本自动降到 116 行
单元测试	39 条全绿，约 41 ms
性能	5 万行 × 2 列：解析 22 ms / 执行 80 ms / 打包 3 ms

4.3 审计的判定口径（可复现性的反向用法）

审计不是”看起来对不对”，而是三态 + 一个哈希：

判定	触发条件
通过	重算值与报告值相对误差 ≤0.1%（报告通常四舍五入到 4 位）
不符	相对误差 >0.1%；或报告**正文**里写的数字与声明的 `value` 不一致
无法复现	引用的 `stepId` 不存在；或该步骤输出里没有声明的 `field`
整体不通过	报告声明的 `expectedArtifactHash` 与重算值不一致

第三种判定是这套设计的价值所在：**“算不出”和”算错了”是两种不同的问题**，混为一谈就无法指导修正。

4.4 已知边界

- 浮点运算在相同 IEEE754 环境一致；**跨 CPU 架构的极端差异未验证**；
- LLM 编译对措辞敏感：换问法可能得到不同计划（但计划一旦确定，执行仍逐位一致）——所以产品把计划做成可导出的文件，这是”同一计划”这一前提的工程回应；
- 支持 ≤5 万行 CSV、13 个算子、单用户、单文件；不做任意代码执行、云端存储、协同编辑。

五、AI 协同过程与工程验证

5.1 协同形态

- 选题、方案、建仓规划在 CherryStudio 完成（4 张原始会话截图见 `prompts/screenshots/`）；
- 代码迭代在本地仓库完成：**27 个 `[iter N]` 提交**，每轮一份记录（我给 AI 的指令原文、AI 产出、我发现了什么问题、怎么改的），见 `prompts/README.md`；
- 规则：一次迭代 = 一个 commit = 一份记录，**Prompt 抄原文，不事后美化。**

5.2 AI 产物的四道机器化验收

#	闸门	拦住什么
1	计划白名单校验	臆造算子、参数类型错、列名不存在、步数超限
2	结论字段绑定校验	编造数字、引用不存在的字段
3	39 条回归测试	行为回退，含 4 条真实翻车用例
4	`verify.mjs` + 界面自校验	执行记录被篡改、包内数据被改

5.3 四个真实翻车（都已修复并补回归测试）

#	现象	根因	教训
1	整页按钮全部失效（骨架在、脚本不执行）	`import { verifyPackage }` 写了两遍	模块级错误只能靠”真跑一次页面”发现，单测覆盖不到
2	筛选与剔除叠加时行号差 1	`detectOutliers` 返回”掩码后下标 + 2”，被当成原始行号	新算子必须验证与既有算子的**叠加顺序**
3	一组均值少了 600	空单元格被当成 0	缺失必须先归一（NA/空串 → null）再统计
4	用户自己导出的包被自家脚本判 FAIL	算子白名单与 `verify.mjs` 白名单脱钩	同一口径只能有一处实现 → 抽出 `canonicalArtifact`

5.4 测试策略

- 纯 Node 运行（`node tests/kernel.test.mjs`），零测试框架依赖；
- 覆盖：CSV 解析、缺失归一、白名单拒绝、破坏性算子确认、可复现哈希、异常点、相关性、分组汇总、Welch t、缺失盘点、时间趋势、直方图、删除清单一致性、空单元格不当 0 的回归、结论引用缺失报错、复现包指纹可重算、审计三态、条件筛选叠加行号回归、文本列筛选、带单位列转数值、界面自校验；
- 边界压测单独脚本：单列 / 全文本 / 空文件 / 只有表头 / 全空值 / 中文列名 / 5 万行。

六、工程指标

指标	数值
代码规模	JavaScript 2,837 行（内核 765 · 计划与审计 461 · 界面 1,312 · 导出 299）
第三方运行依赖	**0**
提交历史	43 个提交，其中 27 个 `[iter N]` 细粒度迭代
测试	39 条，全绿，约 41 ms
性能	5 万行 × 2 列：解析 22 ms / 执行 80 ms / 打包 3 ms
启动	`node serve.mjs`，无 `npm install`、无构建步骤
界面	双模式（分析 / 审计）、五段式工作台、窄屏适配

七、评审维度对照

评分维度	权重	本文档对应章节	仓库中的证据
过程可复现性与演进链路	25%	四、+ 五、5.1	43 个提交 / 27 个 `[iter N]`；`prompts/`（27 轮记录 + 4 张原始会话截图）；README 10 分钟复现
解题有效性与实用价值	25%	一、二、2.1	真实中文数据 + 完整产品闭环 + 审计模式（替别人验）
方法创新性与意图控制	20%	二、2.2–2.6	计划 DSL / 白名单 / confirm 三道闸门 / 结论字段绑定 / 点选确定性入口
工程质量与架构	15%	三、+ 六	分层解耦、零依赖、39 条测试、`verify.mjs` 独立校验
用户体验与产品精细度	15%	三、3.2	结论徽章跳转、数据体检、图表下钻、可撤回剔除、内联提示、计划历史

附录 A：复现命令

```
git clone https://github.com/9568138-cloud/reprolab
cd reprolab
node serve.mjs                                  # → http://127.0.0.1:5173

# 命令行审计（有不通过时退出码 1，可进 CI）
node cli.mjs audit tests/fixtures/audit-case.json \
  --data sample-data/中国70城房价指数_环比.csv

# 单测
node tests/kernel.test.mjs
```

附录 B：文件清单（节选）

```
index.html  serve.mjs  cli.mjs
src/planner.js  src/audit.js  src/styles.css  src/ui/app.js
src/kernel/{csv,hash,stats,ops,profile,index}.js
src/export/{zip,package,verify-runner}.js
tests/kernel.test.mjs  tests/fixtures/audit-case.json
sample-data/（5 份公开真实数据，见 sample-data/README.md）
docs/{技术文档.md, ui-01…ui-09.png}
prompts/{README.md, screenshots/01…04.png}
demo/视频分镜.md
```